

SAVITIBAI PHULE PUNE UNIVERSITY
M.Sc. (Computer Science) Sem-I
Practical Examination (From 2023-2024)
SUBJECT: CS-505-MJP: Lab Course on CS-502-MJ
(Artificial Intelligence)

COMPLETE ANSWERS — SLIP 1 TO SLIP 25

(Each slip contains Q1 and Q2 programs with short explanations)

----- Slip 1 -----

Q1 — Hill Climbing to maximize $f(x) = -x^2 + 4x$

```
# Hill Climbing for  $f(x) = -x^2 + 4x$ 
import random

def f(x): return -x*x + 4*x

def hill_climb(start=None, step=0.1, iters=1000):
    if start is None:
        x = random.uniform(-10,10)
    else:
        x = start
    for i in range(iters):
        left = f(x - step)
        right = f(x + step)
        current = f(x)
        if left > current:
            x -= step
        elif right > current:
            x += step
        else:
            break
    return x, f(x)

if __name__ == "__main__":
    best_x, best_val = hill_climb(start=0.0, step=0.01, iters=10000)
    print(f"Max at x ≈ {best_x:.4f}, f(x) ≈ {best_val:.4f}")
```

Note: Expected maximum at $x=2$, $f(2)=4$.

Q2 — Depth First Search (example graph)

```
# DFS with goal test. Example graph provided; initial=1, goal=8
def dfs(graph, start, goal):
    visited=set()
    stack=[start]
    parent={start: None}
    while stack:
        node = stack.pop()
        if node==goal:
            # reconstruct path
            path=[]
```

```

        cur=goal
        while cur is not None:
            path.append(cur)
            cur=parent[cur]
        return list(reversed(path))
    if node not in visited:
        visited.add(node)
        # push neighbors in reverse order for natural order
        for nbr in reversed(graph.get(node, [])):
            if nbr not in visited:
                parent[nbr]=node
                stack.append(nbr)

    return None

if __name__ == "__main__":
    # sample graph (modify if your question graphic differs)
    graph = {
        1:[2,3],
        2:[4,5],
        3:[6],
        4:[7],
        5:[8],
        6:[],
        7:[],
        8:[]
    }
    path = dfs(graph,1,8)
    print("DFS path from 1 to 8:", path)

```

Note: Replace adjacency with exact graph from exam if different.

----- Slip 2 -----

Q1 — Generate Calendar for given month and year

```

import calendar

def show_calendar(month, year):
    print(calendar.month(year, month))

if __name__ == "__main__":
    m = int(input("Enter month (1-12): "))
    y = int(input("Enter year (e.g. 2025): "))
    show_calendar(m,y)

```

Note: Uses Python's calendar module.

Q2 — DFS (initial=1, goal=7)

```

def dfs_path(graph, start, goal):
    visited=set()
    stack=[start]
    parent={start: None}
    while stack:
        node = stack.pop()
        if node==goal:
            path=[]
            cur=goal
            while cur:
                path.append(cur)
                cur=parent[cur]
            return list(reversed(path))
        if node not in visited:
            visited.add(node)
            for n in reversed(graph.get(node, [])):
                if n not in visited:
                    parent[n]=node
                    stack.append(n)

    return None

if __name__ == "__main__":

```

```
graph = {1:[2,3],2:[4],3:[5,6],4:[7],5:[],6:[7],7:[]}  
print("Path:", dfs_path(graph,1,7))
```

Note: Adjust graph per question.

----- Slip 3 -----

Q1 — Remove punctuations from given string

```
import string

def remove_punct(s):
    return s.translate(str.maketrans('', '', string.punctuation))

if __name__ == "__main__":
    s = input("Enter string: ")
    print("After removing punctuations:")
    print(remove_punct(s))
```

Note: Uses str.translate and string.punctuation.

Q2 — DFS (initial=2, goal=7)

```
# DFS example initial=2 goal=7
def dfs(graph, start, goal):
    visited=set()
    stack=[start]
    parent={start: None}
    while stack:
        node=stack.pop()
        if node==goal:
            path=[]
            cur=goal
            while cur:
                path.append(cur)
                cur=parent[cur]
            return list(reversed(path))
        if node not in visited:
            visited.add(node)
            for n in reversed(graph.get(node, [])):
                if n not in visited:
                    parent[n]=node
                    stack.append(n)
    return None

if __name__ == "__main__":
    graph={1:[2,3],2:[4,5],3:[6],4:[7],5:[],6:[7],7:[]}
    print("Path:", dfs(graph,2,7))
```

----- Slip 4 -----

Q1 — Hangman (console version)

```
import random

def hangman(word):
    word = word.lower()
    guessed=set()
    wrong=0
    max_wrong=6
    display = ['_' if ch.isalpha() else ch for ch in word]
    while wrong < max_wrong and '_' in display:
        print('Word:', ' '.join(display), f"Wrong guesses: {wrong}/{max_wrong}")
        guess = input("Enter a letter: ").lower().strip()
        if not guess or len(guess)!=1:
            print("Enter single letter.")
            continue
        if guess in guessed:
            print("Already guessed.")
            continue
        guessed.add(guess)
        if guess in word:
            for i,ch in enumerate(word):
                if ch==guess:
                    display[i]=guess
```

```

        else:
            wrong+=1
    if '_' not in display:
        print("You won! Word:", word)
    else:
        print("You lost. Word was:", word)

if __name__ == "__main__":
    words = ["python", "hangman", "university", "college"]
    hangman(random.choice(words))

```

Note: Console-based Hangman game.

Q2 — Breadth First Search (initial=1, goal=8)

```

from collections import deque

def bfs(graph, start, goal):
    q=deque([start])
    visited={start}
    parent={start:None}
    while q:
        node=q.popleft()
        if node==goal:
            path=[]
            cur=goal
            while cur:
                path.append(cur)
                cur=parent[cur]
            return list(reversed(path))
        for n in graph.get(node, []):
            if n not in visited:
                visited.add(n)
                parent[n]=node
                q.append(n)
    return None

if __name__ == "__main__":
    graph={1:[2,3], 2:[4,5], 3:[6], 4:[7], 5:[8], 6:[], 7:[], 8:[]}
    print("BFS path 1->8:", bfs(graph,1,8))

```

----- Slip 5 -----

Q1 — Lemmatization using NLTK

```
import nltk
from nltk.stem import WordNetLemmatizer
nltk.download('wordnet', quiet=True)
nltk.download('punkt', quiet=True)

def lemmatize_text(text):
    lemmatizer = WordNetLemmatizer()
    tokens = nltk.word_tokenize(text)
    return ' '.join(lemmatizer.lemmatize(token) for token in tokens)

if __name__=="__main__":
    txt = input("Enter text: ")
    print("Lemmatized:", lemmatize_text(txt))
```

Note: Requires NLTK downloads: 'wordnet' and 'punkt'.

Q2 — BFS (initial=1, goal=8)

```
from collections import deque

def bfs(graph, start, goal):
    q=deque([start]); visited={start}; parent={start:None}
    while q:
        node=q.popleft()
        if node==goal:
            path=[]; cur=goal
            while cur:
                path.append(cur); cur=parent[cur]
            return list(reversed(path))
        for n in graph.get(node, []):
            if n not in visited:
                visited.add(n); parent[n]=node; q.append(n)
    return None

if __name__=="__main__":
    graph={1:[2,3],2:[4],3:[5],4:[6],5:[7,8],6:[],7:[],8:[]}
    print("BFS:", bfs(graph,1,8))
```

----- Slip 6 -----

Q1 — Remove stop words for passage from a text file using NLTK

```
import nltk
nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

def remove_stopwords_from_file(fname):
    with open(fname, 'r', encoding='utf-8') as f:
        text=f.read()
    stop=set(stopwords.words('english'))
    words=word_tokenize(text)
    filtered=[w for w in words if w.lower() not in stop]
    return ' '.join(filtered)

if __name__=="__main__":
    fname = input("Enter filename: ")
    print(remove_stopwords_from_file(fname))
```

Note: Requires NLTK stopwords and punkt.

Q2 — BFS (initial=1, goal=8)

```
from collections import deque

def bfs(graph, start, goal):
```

```

q=deque([start]); visited={start}; parent={start:None}
while q:
    n=q.popleft()
    if n==goal:
        p=[]; cur=goal
        while cur:
            p.append(cur); cur=parent[cur]
        return list(reversed(p))
    for nb in graph.get(n,[]):
        if nb not in visited:
            visited.add(nb); parent[nb]=n; q.append(nb)
return None

if __name__=="__main__":
    graph={1:[2,3],2:[4],3:[5,6],4:[7],5:[8],6:[],7:[],8:[]}
    print("Path:", bfs(graph,1,8))

```

----- Slip 7 -----

Q1 — Tic-Tac-Toe using Alpha-Beta Pruning

```
# Simple Tic-Tac-Toe with minimax + alpha-beta
import math
def winner(board):
    lines=[(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
    for a,b,c in lines:
        if board[a]==board[b]==board[c] and board[a]!=' ':
            return board[a]
    if ' ' not in board:
        return 'Draw'
    return None

def minimax(board, player, alpha, beta):
    w=winner(board)
    if w=='X': return (1,None)
    if w=='O': return (-1,None)
    if w=='Draw': return (0,None)
    if player=='X':
        best=(-math.inf,None)
        for i in range(9):
            if board[i]==' ':
                board[i]=player
                val,_=minimax(board,'O',alpha,beta)
                board[i]=' '
                if val>best[0]: best=(val,i)
                alpha=max(alpha,val)
                if beta<=alpha: break
        return best
    else:
        best=(math.inf,None)
        for i in range(9):
            if board[i]==' ':
                board[i]=player
                val,_=minimax(board,'X',alpha,beta)
                board[i]=' '
                if val<best[0]: best=(val,i)
                beta=min(beta,val)
                if beta<=alpha: break
        return best

# Simple console play where AI is X
if __name__=="__main__":
    board=[' ']*9
    human='O'; ai='X'
    while True:
        print('Board:')
        for r in range(3):
            print(board[3*r:3*r+3])
        if winner(board): break
        pos=int(input("Enter pos 0-8: "))
        if board[pos]==' ':
            board[pos]=human
        else:
            print("Invalid")
            continue
        if winner(board): break
        val,ai_move=minimax(board,ai,-math.inf,math.inf)
        board[ai_move]=ai
    print("Result:", winner(board))
```

Note: Alpha-beta pruning minimax implementation.

Q2 — Simple Chatbot (rule-based)

```
def chatbot():
    print("Chatbot: Hello! Type 'bye' to exit.")
    while True:
        s = input("You: ").lower()
        if s in ('bye','exit','quit'):
```



```

        print("Chatbot: Bye!")
        break
    if "hello" in s or "hi" in s:
        print("Chatbot: Hello, how can I help?")
    elif "assignment" in s or "lab" in s:
        print("Chatbot: Lab schedules are on the department notice board.")
    elif "who are you" in s:
        print("Chatbot: I'm a simple AI chatbot for demo.")
    else:
        print("Chatbot: Sorry, I didn't get that.")

if __name__=="__main__":
    chatbot()

```

----- Slip 8 -----

Q1 — Count upper and lower case letters

```

def count_case(s):
    upper=sum(1 for c in s if c.isupper())
    lower=sum(1 for c in s if c.islower())
    return upper, lower

if __name__=="__main__":
    s=input("Enter string: ")
    u, l = count_case(s)
    print("Upper:",u,"Lower:",l)

```

Note: Counts uppercase and lowercase letters.

Q2 — Solve Tic-Tac-Toe (reuse minimax)

Use minimax from Slip 7; implement same AI for solving Tic-Tac-Toe.

----- Slip 9 -----

Q1 — 8-puzzle using A* algorithm (Manhattan heuristic)

```
import heapq

def manhattan(state, goal):
    dist=0
    for i,val in enumerate(state):
        if val==0: continue
        j=goal.index(val)
        dist += abs(i//3 - j//3) + abs(i%3 - j%3)
    return dist

def astar(start, goal):
    start=tuple(start); goal=tuple(goal)
    openp=[(manhattan(start, list(goal)),0,start, None)]
    heapq.heapify(openp)
    g={start:0}
    parent={start:None}
    while openp:
        f,gcost,state,_ = heapq.heappop(openp)
        if state==goal:
            path=[]; cur=state
            while cur:
                path.append(list(cur))
                cur=parent[cur]
            return list(reversed(path))
        zero = state.index(0)
        x,y = zero//3, zero%3
        moves=[]
        for dx,dy in [(-1,0),(1,0),(0,-1),(0,1)]:
            nx,ny = x+dx, y+dy
            if 0<=nx<3 and 0<=ny<3:
                lst=list(state)
                idx=nx*3+ny
                lst[zero], lst[idx] = lst[idx], lst[zero]
                moves.append(tuple(lst))
        for m in moves:
            ng = gcost+1
            if m not in g or ng < g[m]:
                g[m]=ng
                parent[m]=state
                heapq.heappush(openp, (ng+manhattan(m, list(goal)),ng,m,None))
    return None

if __name__=="__main__":
    start = [1,2,3,4,0,6,7,5,8]
    goal = [1,2,3,4,5,6,7,8,0]
    path = astar(start, goal)
    print("Steps:", len(path)-1)
    for p in path:
        print(p[0:3]); print(p[3:6]); print(p[6:9]); print()
```

Note: A* with Manhattan distance heuristic.

Q2 — Water Jug (5 & 7, target 4 in second jug)

```
from collections import deque

def water_jug(capA, capB, targetB):
    start=(0,0)
    q=deque([start]); visited={start}; parent={start:None}
    while q:
        a,b=q.popleft()
        if b==targetB:
            path=[]
            cur=(a,b)
            while cur:
                path.append(cur)
                cur=parent[cur]
            return list(reversed(path))
```

```

        moves.append((capA,b))
        moves.append((a, capB))
        moves.append((0,b))
        moves.append((a,0))
        transfer=min(a, capB-b)
        moves.append((a-transfer, b+transfer))
        transfer=min(b, capA-a)
        moves.append((a+transfer, b-transfer))
        for m in moves:
            if m not in visited:
                visited.add(m); parent[m]=(a,b); q.append(m)
    return None

if __name__=="__main__":
    path = water_jug(5,7,4)
    print("Solution path:", path)

```

----- Slip 10 -----

Q1 — Cryptarithmic $TWO + TWO = FOUR$ (backtracking)

```

import itertools

def solve():
    letters = set("TW O F U R".replace(" ", ""))
    letters = list(letters)
    for perm in itertools.permutations('0123456789', len(letters)):
        assign = dict(zip(letters, perm))
        if assign['T']=='0' or assign['F']=='0': continue
        def num(s): return int(''.join(assign[c] for c in s))
        if num("TWO") + num("TWO") == num("FOUR"):
            return assign, num("TWO"), num("FOUR")
    return None

if __name__=="__main__":
    sol = solve()
    print(sol)

```

Note: Backtracking search assigning digits to letters.

Q2 — Simple Chatbot

```

# reuse Slip 7 chatbot or a slightly extended one
responses = {
    "hello": "Hi there! I'm your campus bot.",
    "time": "Check your system clock.",
    "lab": "Lab timings are Mon-Fri 10-4."
}

def bot():
    print("Type 'bye' to exit.")
    while True:
        s=input("You: ").lower()
        if s in ('bye', 'exit'): break
        print("Bot:", responses.get(s, "Sorry, I don't know. "))

if __name__=="__main__":
    bot()

```

----- Slip 11 -----

Q1 — Mean-end analysis for transforming lowercase string to another (simple BFS of edits)

```
from collections import deque

def mean_end_transform(start, goal):
    visited={start}
    q=deque([(start, [start])])
    while q:
        s, path = q.popleft()
        if s==goal: return path
        for i in range(len(s)):
            for ch in 'abcdefghijklmnopqrstuvwxyz':
                if ch==s[i]: continue
                ns = s[:i]+ch+s[i+1:]
                if ns not in visited:
                    visited.add(ns)
                    q.append((ns, path+[ns]))
    return None

if __name__=="__main__":
    start = input("Start string: ")
    goal = input("Goal string: ")
    p = mean_end_transform(start, goal)
    print("Steps:", p)
```

Note: Uses BFS over string-edit space.

Q2 — Water Jug (4 & 3, target 2 in second jug)

```
# reuse water_jug function
from collections import deque

def water_jug(capA, capB, targetB):
    start=(0,0); q=deque([start]); visited={start}; parent={start:None}
    while q:
        a,b=q.popleft()
        if b==targetB:
            path=[]; cur=(a,b)
            while cur:
                path.append(cur); cur=parent[cur]
            return list(reversed(path))
        moves=[]
        moves.append((capA,b)); moves.append((a,capB)); moves.append((0,b)); moves.append((a,0))
        transfer=min(a,capB-b); moves.append((a-transfer,b+transfer))
        transfer=min(b,capA-a); moves.append((a+transfer,b-transfer))
        for m in moves:
            if m not in visited:
                visited.add(m); parent[m]=(a,b); q.append(m)
    return None

if __name__=="__main__":
    print(water_jug(4,3,2))
```

----- Slip 12 -----

Q1 — Calendar (same as Slip 2)

```
import calendar
m=int(input("Month: ")); y=int(input("Year: "))
print(calendar.month(y,m))
```

Note: Generate monthly calendar.

Q2 — Simulate 4-Queens problem (backtracking)

```
def solve_n_queens(n=4):
    solutions=[]
    def backtrack(col, cols, diag1, diag2, board):
        if col==n:
```

```

        solutions.append(board.copy()); return
    for row in range(n):
        if row in cols or (row+col) in diag1 or (row-col) in diag2: continue
        cols.add(row); diag1.add(row+col); diag2.add(row-col)
        board.append(row)
        backtrack(col+1, cols, diag1, diag2, board)
        board.pop(); cols.remove(row); diag1.remove(row+col); diag2.remove(row-col)
    backtrack(0, set(), set(), set(), [])
    return solutions

if __name__=="__main__":
    sols=solve_n_queens(4)
    print("Number of solutions:", len(sols))
    for s in sols: print(s)

```

----- Slip 13 -----

Q1 — Mini-Max Algorithm (tic-tac-toe)

Mini-Max function example for 3x3 tic-tac-toe returning best move for player
Use minimax implementation from Slip 7

Note: Minimax used for two-player zero-sum games.

Q2 — Simulate 8-Queens (backtracking)

```
def solve_n_queens(n=8):
    solutions=[]
    def backtrack(col, cols, diag1, diag2, board):
        if col==n:
            solutions.append(board.copy()); return
        for row in range(n):
            if row in cols or (row+col) in diag1 or (row-col) in diag2: continue
            cols.add(row); diag1.add(row+col); diag2.add(row-col)
            board.append(row)
            backtrack(col+1, cols, diag1, diag2, board)
            board.pop(); cols.remove(row); diag1.remove(row+col); diag2.remove(row-col)
        backtrack(0, set(), set(), set(), [])
    return solutions

if __name__=="__main__":
    sols=solve_n_queens(8)
    print("Solutions found:", len(sols))
    print("One solution:", sols[0])
```

----- Slip 14 -----

Q1 — Sort sentence in alphabetical order

```
def sort_sentence(s):
    words = s.split()
    words.sort(key=lambda w: w.lower())
    return ' '.join(words)
```

```
if __name__=="__main__":
    s=input("Enter sentence: ")
    print(sort_sentence(s))
```

Note: Sorts words ignoring case.

Q2 — Simulate N-Queens (generic n)

Reuse the n-queens solver; ask user for n
n=int(input("Enter n for n-queens: "))

```
def solve(n):
    solutions=[]
    def backtrack(col, cols, d1, d2, b):
        if col==n:
            solutions.append(b.copy()); return
        for row in range(n):
            if row in cols or (row+col) in d1 or (row-col) in d2: continue
            cols.add(row); d1.add(row+col); d2.add(row-col)
            b.append(row); backtrack(col+1, cols, d1, d2, b)
            b.pop(); cols.remove(row); d1.remove(row+col); d2.remove(row-col)
        backtrack(0, set(), set(), set(), [])
    return solutions

sols=solve(n)
print("Count:", len(sols))
if sols: print("One sol:", sols[0])
```

----- Slip 15 -----

Q1 — Monkey Banana Problem (BFS sketch)

```
from collections import deque

def monkey_banana():
    start=('left', False, 'left')
    q=deque([start]); parent={start:None}
    while q:
        s=q.popleft()
        monkey, hasb, box = s
        if hasb:
            path=[]; cur=s
            while cur:
                path.append(cur); cur=parent[cur]
            return list(reversed(path))
        for mpos in ('left','right'):
            if mpos!=monkey:
                ns=(mpos, hasb, box)
                if ns not in parent:
                    parent[ns]=s; q.append(ns)
        if monkey==box:
            nb = 'right' if box=='left' else 'left'
            ns=(nb, hasb, nb)
            if ns not in parent:
                parent[ns]=s; q.append(ns)
        if box=='right' and monkey=='right' and not hasb:
            ns=(monkey, True, box)
            if ns not in parent:
                parent[ns]=s; q.append(ns)
    return None

if __name__=="__main__":
    print(monkey_banana())
```

Note: BFS on a simple state graph for monkey and banana.

Q2 — Iterative Deepening DFS (IDDFS)

```
def dls(graph, start, goal, limit):
    if start==goal: return [start]
    if limit<=0: return None
    for n in graph.get(start,[]):
        path = dls(graph, n, goal, limit-1)
        if path:
            return [start]+path
    return None

def iddfs(graph, start, goal, max_depth=10):
    for depth in range(max_depth+1):
        path = dls(graph, start, goal, depth)
        if path:
            return path
    return None

if __name__=="__main__":
    g={'S':['A','B'], 'A':['C'], 'B':['D','G'], 'C':[], 'D':[], 'G':[]}
    print(iddfs(g,'S','G',10))
```

----- Slip 16 -----

Q1 — Tower of Hanoi

```
def tower_of_hanoi(n, src, aux, dest, moves):
    if n==1:
        moves.append((src,dest))
        return
    tower_of_hanoi(n-1, src, dest, aux, moves)
    moves.append((src,dest))
```

```
        tower_of_hanoi(n-1, aux, src, dest, moves)

if __name__=="__main__":
    n=int(input("Enter number of disks: "))
    moves=[]
    tower_of_hanoi(n, 'A', 'B', 'C', moves)
    print("Moves:", len(moves))
    for m in moves: print(m)
```

Note: Recursive solution for Tower of Hanoi.

Q2 — Tic-Tac-Toe (15 marks) — reuse minimax from Slip 7

Use minimax implementation from Slip 7.

----- Slip 17 -----

Q1 — Hill Climbing (same as Slip 1)

```
import random

def f(x): return -x*x + 4*x

def hill_climb(start=1.0, step=0.01, iters=10000):
    x=start
    for i in range(iters):
        left=f(x-step); right=f(x+step); cur=f(x)
        if left>cur: x-=step
        elif right>cur: x+=step
        else: break
    return x,f(x)
print(hill_climb(0.0))
```

Note: Hill climbing example.

Q2 — A* algorithm on graph (Start A, Goal G) — example with heuristic

```
import heapq

def a_star(graph, heur, start, goal):
    openp=[(heur[start],0,start)]
    parent={start:None}; g={start:0}
    while openp:
        f,curr_g,node = heapq.heappop(openp)
        if node==goal:
            path=[]; cur=node
            while cur:
                path.append(cur); cur=parent[cur]
            return list(reversed(path))
        for neigh,w in graph.get(node,[]):
            ng = curr_g + w
            if neigh not in g or ng < g[neigh]:
                g[neigh]=ng; parent[neigh]=node
                heapq.heappush(openp,(ng+heur[neigh], ng, neigh))
    return None

if __name__=="__main__":
    graph = {
        'A':[( 'B',1),('C',4)],
        'B':[( 'D',2),('E',5)],
        'C':[( 'F',1)],
        'D':[( 'G',7)],
        'E':[( 'G',2)],
        'F':[( 'G',3)],
        'G':[]
    }
    heur = {'A':7, 'B':6, 'C':5, 'D':3, 'E':2, 'F':3, 'G':0}
    print("A* path:", a_star(graph,heur,'A','G'))
```

----- Slip 18 -----

Q1 — Remove stop words from file (same as Slip 6)

See Slip 6 remove_stopwords_from_file implementation.

Note: Same as Slip 6.

Q2 — A* for placement arrangement (skeleton)

```
import heapq

def a_star_placement(start_state, goal_test, get_neighbors, heuristic):
    openp=[(heuristic(start_state),0,start_state)]
    g={start_state:0}; parent={start_state:None}
    while openp:
        f,curr_g,state = heapq.heappop(openp)
```

```

    if goal_test(state):
        path=[]; cur=state
        while cur:
            path.append(cur); cur=parent[cur]
        return list(reversed(path))
    for neigh,cost in get_neighbors(state):
        ng = curr_g + cost
        if neigh not in g or ng<g[neigh]:
            g[neigh]=ng; parent[neigh]=state
            heapq.heappush(openp,(ng+heuristic(neigh), ng, neigh))
    return None

```

print("Provide assumed dimensions and implement the skeleton for full credit.")

Note: Skeleton for A* placement problem; explain heuristic and state representation.

----- Slip 19 -----

Q1 — Hangman (same as Slip 4)

Use Hangman code from Slip 4.

Note: Console Hangman.

Q2 — A* algorithm (general) — reuse Slip 17 A* example

Use A* code from Slip 17 with appropriate graph and heuristic.

----- Slip 20 -----

Q1 — Build a bot which provides all information related to you in college

```
info = {
    "name": "Your Name",
    "department": "Computer Science",
    "year": "M.Sc. Sem I",
    "email": "yourcollege@edu",
    "lab_timings": "Mon-Fri 10:00-17:00",
    "office": "Block A, Room 12"
}

def college_bot():
    print("Type 'bye' to exit")
    while True:
        q=input("Ask: ").lower()
        if q in ('bye','exit'): break
        for k in info:
            if k in q:
                print(info[k]); break
        else:
            print("I can provide: name, department, year, email, lab_timings, office")

if __name__=="__main__":
    college_bot()
```

Note: Simple rule-based information bot; replace values with real student info.

Q2 — Mini-Max Algorithm (restate or reuse Slip 13)

Use minimax implementation from Slip 7 for decision making.

----- Slip 21 -----

Q1 — Remove punctuations from the given string

```
import string

def remove_punct(s): return s.translate(str.maketrans('', '', string.punctuation))
print(remove_punct(input('Enter string: ')))
```

Note: Same as Slip 3.

Q2 — Cryptarithmic GO + TO = OUT

```
import itertools

def solve_cryptarithmic():
    letters = list(set('GOTU'))
    for perm in itertools.permutations('0123456789', len(letters)):
        assign = dict(zip(letters, perm))
        if assign['G']=='0' or assign['T']=='0' or assign['O']=='0': continue
        def num(s): return int(''.join(assign[c] for c in s))
        if num("GO") + num("TO") == num("OUT"):
            return assign, num("GO"), num("TO"), num("OUT")
    return None

if __name__=="__main__":
    print(solve_cryptarithmic())
```

Note: Adjust letters set if needed.

----- Slip 22 -----

Q1 — Alpha-Beta Pruning implementation

Use minimax with alpha-beta pruning code from Slip 7.

Note: Alpha-beta reduces nodes explored by pruning.

Q2 — Simple Chatbot (same as Slip 7)

Use chatbot from Slip 7.

----- Slip 23 -----

Q1 — Tower of Hanoi (same as Slip 16)

Use Tower of Hanoi code from Slip 16.

Note: Recursive solution.

Q2 — SEND + MORE = MONEY (cryptarithmic)

import itertools

```
def solve_send_more_money():
    letters = list(set('SENDMOREY'))
    for perm in itertools.permutations('0123456789', len(letters)):
        assign = dict(zip(letters, perm))
        if assign['S']=='0' or assign['M']=='0': continue
        def val(word): return int(''.join(assign[ch] for ch in word))
        if val('SEND') + val('MORE') == val('MONEY'):
            return assign, val('SEND'), val('MORE'), val('MONEY')
    return None

if __name__=="__main__":
    sol = solve_send_more_money()
    print(sol)
```

Note: Classic cryptarithmic; known solution exists.

----- Slip 24 -----

Q1 — Sort sentence in alphabetical order

```
s=input('Enter sentence: ')
print(' '.join(sorted(s.split(), key=lambda w:w.lower())))
```

Note: Same as Slip 14.

Q2 — CROSS + ROADS = DANGER (cryptarithmic skeleton)

import itertools

```
def solve_cross_roads():
    letters = list(set('CROSSROADS DANGER'.replace(' ', '')))
    for perm in itertools.permutations('0123456789', len(letters)):
        assign = dict(zip(letters, perm))
        if assign.get('C','0')=='0' or assign.get('R','0')=='0' or assign.get('D','0')=='0':
            continue
        def val(w): return int(''.join(assign[ch] for ch in w))
        if val('CROSS') + val('ROADS') == val('DANGER'):
            return assign
    return None

if __name__=="__main__":
    print('Use backtracking; may take long to compute.')
```

Note: Skeleton code for cryptarithmic.

----- Slip 25 -----

Q1 — Build a bot which provides all the information related to you in college (same as Slip 20)

Use Slip 20 bot code and fill in actual details.

Note: Rule-based bot.

Q2 — Solve 8-puzzle problem (same as Slip 9)

Use A* 8-puzzle solver from Slip 9.

COMMON VIVA (Q3) — Suggested Short Q&A;

Q: What is DFS and where is it used?

A: Depth First Search explores as deep as possible along a branch before backtracking. Used in pathfinding, topological sort, solving puzzles.

Q: What is the heuristic in A*?

A: Heuristic estimates cost from current node to goal; must be admissible (not overestimate) to guarantee optimality.

Q: Difference between informed and uninformed search?

A: Informed uses problem-specific knowledge (heuristic), uninformed does not.

Q: How does alpha-beta pruning optimize minimax?

A: It prunes branches that won't affect final decision by keeping alpha (best for maximizer) and beta (best for minimizer) bounds.

Q: For NLP tasks, why use lemmatization and stop-word removal?

A: Lemmatization reduces words to base forms for normalization; stop-word removal reduces noise and dimensionality.